Reproducible Data Science Projects

An Instructor's Practical Framework

Table of contents

1 Introduction **2**
- 1.1 Two complementary dimensions of reproducibility . . . 2
- 1.2 Reproducible research and scientific communication . . . 3
- 1.3 Relationship to CRISP-DM . . . 4

2 Organization of a Reproducible Research Project **4**
- 2.1 The repository as the unit of reproducibility . . . 4
- 2.2 A minimal directory structure . . . 5
- 2.3 Source files and generated artifacts . . . 6
- 2.4 Naming and paths . . . 6

3 Computational Notebooks **7**
- 3.1 The role of Quarto notebooks . . . 7
- 3.2 Literate programming and the computational "How" . . . 7
- 3.3 One analytical responsibility per notebook . . . 7
- 3.4 A standard notebook anatomy . . . 8
- 3.5 Independent execution and explicit dependencies . . . 8
- 3.6 Computational transparency and reproducibility . . . 8

4 Research Reports and Scientific Papers **9**
- 4.1 Research reports as methodological documentation . . . 9
- 4.2 Scientific papers as communication of research contributions . . . 9
- 4.3 Complementary responsibilities . . . 10
- 4.4 Relationship to reporting milestones . . . 10

5 Modular and Automated Workflows **11**
- 5.1 Analytical versus software modularity . . . 11
- 5.2 Reusable R functions and `src/` . . . 11
- 5.3 Executable scripts and `scripts/` . . . 12

5.4 From interactive analysis to an automated pipeline 12
5.5 Reproducibility checks 12

6 Educational Use of the Framework **13**
6.1 Instructor responsibilities and intended competencies 13
6.2 Framework versus classroom implementation 13

7 Concluding Principles **14**

1 Introduction

A Data Science research project involves more than applying statistical methods to a dataset. It requires formulating a meaningful question, identifying and evaluating relevant information, preparing data for analysis, interpreting results, and communicating findings. A reproducible project makes both the reasoning and the computations behind these activities accessible to others.

This guide presents a practical framework for organizing reproducible research in Data Science projects. It integrates research documentation, literate programming, computational environment management, version control, and modular project organization. Its aim is to make research decisions understandable, computational results reproducible, and investigations easier to continue across changes in research personnel.

The immediate audience is instructors coordinating research-oriented Data Science courses. The guide explains the rationale for the framework and the responsibilities of its components; it is not a student tutorial or a prescribed sequence of assignments. Separate student-facing instructions may translate these principles into concrete activities. The framework may also provide a basis for subsequent educational research and publication, but its immediate purpose is to support a shared understanding among instructors.

1.1 Two complementary dimensions of reproducibility

In this guide we recognize two complementary dimensions of reproducibility:

- **Computational reproducibility:** is the ability to obtain the same computational results by executing the documented procedures, with the same inputs, in a reproducible computational environment. It depends on the availability of data, code, dependencies, configuration, and execution instructions. When randomness is involved, seeds and other relevant settings must also be controlled. However, some computations may still vary across platforms or software versions.

- **Methodological reproducibility:** refers to making the research design and analytical decisions sufficiently explicit that another researcher can understand, scrutinize, and apply the same procedures. This includes the formulation of research questions, the choice of observations and variables, the treatment of missing data, the selection of statistical methods, and the interpretation of results. This term is used here as a practical organizing distinction; terminology varies across research fields.

We should highlight that these dimensions are complementary, but neither guarantees the other. A script may reproduce a numerical result without explaining why the calculation was appropriate. Conversely, a carefully justified report may be impossible to reproduce computationally if its code or data are unavailable.

Dimension	Guiding question	Principal evidence
Computational reproducibility	Can the calculations be executed again?	Data, executable code, dependencies, configuration, instructions
Methodological reproducibility	Can the research decisions be understood and evaluated?	Questions, assumptions, decision criteria, justifications, limitations

Reproducibility is distinct from *replication*, which commonly refers to investigating whether a result can be obtained again using new data or an independently conducted study. Usage of these terms differs across disciplines.

1.2 Reproducible research and scientific communication

Reproducible Research is the overarching objective of this framework. The computational and methodological dimensions above concern whether the work can be reconstructed and examined. Scientific communication serves a related but distinct purpose: exposing selected findings and their supporting evidence to a wider research community.

Three kinds of documents have complementary responsibilities:

Artifact	Primary question	Principal purpose
Computational notebook	*How* were the results computed?	Explain and execute analytical procedures
Research report	*What* was investigated, and *why* were decisions made?	Preserve methodological knowledge and support research continuity

Artifact	Primary question	Principal purpose
Scientific paper	*What contribution* do the findings make, and *what evidence supports it*?	Communicate and substantiate selected findings in their scientific context

These are distinctions of purpose, not exclusive boundaries. A paper must contain sufficient methodological information for independent evaluation and must meet applicable reproducibility and reporting standards. A report should identify the computational evidence supporting its findings. A notebook must record the precise decisions needed to execute an analysis, even when their broader justification is developed in the report.

Scientific dissemination and scrutiny should not be treated as a third type of reproducibility. Likewise, reproducibility does not by itself establish scientific validity: an inappropriate method can be executed perfectly consistently.

1.3 Relationship to CRISP-DM

The Cross-Industry Standard Process for Data Mining (CRISP-DM) provides a useful reference for organizing Data Science work: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. A research project, however, need not follow these phases literally.

In an academic investigation, researchers may formulate their own questions rather than translate an organizational client's objectives into analytical tasks. Available datasets may constrain the questions that can be answered, and exploration may motivate revisions to the original research design. Accordingly, the approach developed here is *CRISP-DM inspired*, rather than a strict implementation of CRISP-DM.

The methodology describes the research process; the repository architecture describes how to document and implement that process. They need not have identical structures. In particular, the organization of computational notebooks should reflect coherent analytical responsibilities, not necessarily the sequence of reporting deadlines.

2 Organization of a Reproducible Research Project

2.1 The repository as the unit of reproducibility

A single notebook is rarely sufficient to reproduce an entire investigation. Reproduction may also require input datasets, reusable code, package versions, configuration files, and instructions. The *repository* is therefore the natural unit for organizing the project's reproducible materials.

The repository should make it possible to determine:

1. Which inputs were used, and where they came from.
2. Which procedures generated each analytical result.
3. Which outputs were produced, and where they are located.
4. Which assumptions and decisions informed the analysis.
5. How another person can execute the relevant computations.

Version control, ordinarily with Git, records the evolution of source files and supports collaboration. A repository alone does not guarantee reproducibility: external data, software environments, file paths, and execution order still require attention.

2.2 A minimal directory structure

A small project can begin without separate directories for reusable functions or automation scripts:

```
project/
|-- README.md
|-- .gitignore
|-- renv.lock
|-- renv/
|   |-- activate.R
|-- data/
|   |-- raw/
|   |-- processed/
|-- notebooks/
|-- quarto/
|   |-- reports/
|   |-- _quarto.yaml
|-- docs/
    |-- reports/
```

This example assumes a Quarto project rooted at `project/quarto/`, with `project.output-dir: ../docs`; Quarto generally preserves source-relative subdirectories within the output directory. The exact output location should be checked for the selected formats and Quarto version.

The responsibilities of the different components of the repository are summarized in the following table:

Location	Responsibility
`README.md`	Explain the project, prerequisites, and reproduction procedure
`quarto/_quarto.yaml`	Configure Quarto project rendering
`data/raw/`	Hold original input data, unchanged where redistribution is permitted
`data/processed/`	Hold the processed data that will be used in the research project
`notebooks/`	Hold executable analytical documents
`quarto/reports/`	Hold report source documents
`docs/`	Hold generated, distributable outputs

A later project may add `src/` and `scripts/`. These should be introduced when they solve an actual organizational problem, not merely to imitate a large software project.

2.3 Source files and generated artifacts

Source files are the materials edited by researchers: `.qmd` documents, R code, configuration, and documentation. *Generated artifacts* are outputs created from those sources: rendered PDF or HTML reports, figures, tables, and prepared datasets.

Separating sources from generated outputs reduces ambiguity. A PDF report belongs in a publication or distribution location; the `.qmd` document that generates it belongs with the project's editable source materials. When practical, generated outputs should be reproducible from the committed sources rather than manually modified after rendering.

Not every generated file must be tracked in Git. The decision depends on whether the file is a required deliverable, how easily it can be regenerated, its size, and applicable data-sharing restrictions. Large or sensitive raw datasets may require an access procedure rather than inclusion in the repository.

2.4 Naming and paths

Use descriptive, stable filenames such as `data_quality.qmd` or `missingness_profile.qmd`. Numeric prefixes may help when a meaningful sequence exists, but should not imply that notebooks require a shared interactive session.

Prefer project-relative paths over machine-specific absolute paths. A project should not depend on a particular student's home directory or on files manually placed outside the documented repository structure. In R, tools such as `here` can help construct paths from a consistent project root, provided the root is configured and understood.

3 Computational Notebooks

3.1 The role of Quarto notebooks

A Quarto `.qmd` document combines prose, executable R code, and computational outputs. In this guide, a *notebook* is an executable analytical document whose main responsibility is to perform and explain a coherent computational task.

A notebook is more than a collection of R commands. It should state what it is trying to establish, make its inputs explicit, document its procedure, present its outputs, and explain what those outputs imply for the investigation.

3.2 Literate programming and the computational "How"

The use of computational notebooks draws on the philosophy of *literate programming*: executable instructions are accompanied by explanations written for human readers. In this framework, a notebook is a technical account of *how* an analytical task was carried out, not a second scientific report.

Its prose should explain inputs, operations, parameters, outputs, and relevant implementation choices. It may provide concise analytical context, but extended arguments about the scientific appropriateness of a method belong in the research report. For example, a notebook should identify the precise missing-data rule applied and show its consequences; the report should justify that rule in relation to the research question and the available evidence.

Literate documentation improves intelligibility and computational transparency. It does not, on its own, guarantee reproducibility: data access, package versions, execution conditions, and dependencies must also be addressed at the repository level.

3.3 One analytical responsibility per notebook

A useful principle, inspired by the Unix philosophy, is to *do one thing and do it well*. Applied to notebooks, this means that each document should have a recognizable analytical purpose. A notebook might examine the structure of a dataset, assess data quality, or construct a prepared dataset. Each task may involve several functions and calculations.

The appropriate unit of modularity is a *coherent analytical task*, not an individual R statement. Creating a separate notebook for every minor calculation would introduce unnecessary fragmentation.

A notebook may be reused or revisited during multiple reporting periods. Conversely, a single report may draw on several notebooks. Thus notebook boundaries should not be dictated solely by reporting deadlines.

3.4 A standard notebook anatomy

A consistent internal structure makes notebooks easier to review and reproduce:

Element	Question addressed
Objective	What does this notebook seek to establish or produce?
Inputs	Which data, parameters, and dependencies are required?
Procedure	Which operations are performed, and in what order?
Outputs	Which tables, figures, datasets, or results are produced?
Interpretation	What do the outputs show about this computational task, and what checks or limitations matter for their use?

This is a conceptual template, not a requirement to use exactly five headings in every document. The explanation should be proportional to the complexity and significance of the task.

3.5 Independent execution and explicit dependencies

A fundamental rule is that *each notebook should run in a fresh R session*, without depending on objects left in memory by earlier interactive work. It may depend on files produced by another step, but such dependencies must be documented and reproducible.

For example, a notebook should load its own input dataset rather than assume an object called `data` already exists in the global environment. If a processed dataset is required, the project must explain how that dataset is created.

This principle helps prevent a common notebook failure: a document that appears to work interactively but fails when executed from beginning to end. A useful validation practice is to render the notebook in a clean session and inspect the resulting outputs and errors.

3.6 Computational transparency and reproducibility

Computational transparency means that the reader can understand which computations were performed. *Computational reproducibility* additionally requires that those computations can actually be executed again under documented conditions.

Readable code and explanatory text support transparency. Explicit dependencies, accessible inputs, controlled randomness, and clean execution support reproducibility. Neither should be treated as a substitute for the other.

4 Research Reports and Scientific Papers

4.1 Research reports as methodological documentation

The primary audience for a *research report* is a researcher who needs to understand, evaluate, join, or continue the investigation. Its purpose is to preserve the methodological knowledge accumulated by the project, rather than to announce a selected contribution to the scientific literature.

A report should explain *what* was investigated and *why* the principal decisions were made. It should document the research questions, available data, methodological design, alternatives considered where relevant, results obtained, limitations, unresolved questions, and possible next steps. It should be sufficiently detailed to support research continuity without becoming an exhaustive chronological diary of every action.

For example, a report may explain why observations with missing values were excluded, what alternatives were considered, and how that decision affects the interpretation of the results. A supporting notebook can show exactly how missingness was quantified, which rows were excluded, and how the resulting dataset was checked.

This role is particularly valuable when successive groups of students or collaborators contribute to the same project. The report should reduce the effort required for a new researcher to understand the current state of the investigation and participate meaningfully.

4.2 Scientific papers as communication of research contributions

A *scientific paper* addresses a wider disciplinary audience. Its purpose is to identify, contextualize, and substantiate a contribution to scientific knowledge. It emphasizes the relevant literature, the question being answered, the novelty or significance of the findings, the evidence supporting the claims, and the implications and limitations of the contribution.

A paper is therefore organized around a *scientific contribution*, while a research report is organized around the *investigation*. A project may document unsuccessful experiments, abandoned alternatives, and open questions that are useful for its continuation but peripheral to a focused publication. One project may support multiple papers, or no publication at all.

The paper must still describe its methods in sufficient detail for scientific scrutiny and, where applicable, reproducibility. Essential information cannot be omitted merely because a project report exists. The report, however, can preserve a more complete account of the decisions and

history needed to continue the broader investigation. Supporting code, data, and reports can be made available as supplementary materials when appropriate and permitted.

4.3 Complementary responsibilities

The complementary responsibilities of computational notebooks, reports, and papers can be summarized as follows:

Concern	Report	Notebook	Scientific paper
Main audience	Current and future collaborators	People executing or reviewing computations	Disciplinary research community
Research question	Explain the project questions and their evolution	State the analytical task being implemented	Focus on the question behind the contribution
Methodological choices	Explain and justify decisions and relevant alternatives	Record the precise choices implemented	Describe and justify choices essential to evaluating the claims
Results	Preserve relevant findings, including inconclusive ones	Produce detailed outputs and checks	Select and present evidence supporting the contribution
Interpretation	Explain implications for the continuing project	Interpret task-specific outputs and diagnostics	Situate findings within scientific knowledge
Reproducibility	Link decisions and findings to supporting materials	Supply executable computational evidence	Supply sufficient methodological detail and access to supporting evidence as required

The division is one of *emphasis, audience, and purpose*, not an absolute separation of subject matter. In particular, methodological justification is central to the report, but a scientific paper must remain independently assessable.

4.4 Relationship to reporting milestones

Reports may be organized around deadlines, project reviews, or course stages, but their lasting value lies in preserving an intelligible account of the investigation. Computational notebooks need not follow the same schedule. One report can synthesize several notebooks, and one notebook can support multiple reports or papers.

To preserve traceability, reports and papers should identify the relevant notebook or reproducible source for major computational findings. Where feasible, tables and figures should be generated from the same underlying computations rather than manually transcribed.

5 Modular and Automated Workflows

5.1 Analytical versus software modularity

Two forms of modularity should be distinguished:

- **Analytical modularity:** divides the investigation into coherent tasks, each represented by a notebook or another well-defined analytical component.
- **Software modularity:** separates reusable computational logic into functions and modules that can be called from multiple places.

Analytical modularity can be learned and practiced before software modularity. A small project may reasonably contain most of its code directly in notebooks. Repeated or complex computations can later be extracted into functions.

5.2 Reusable R functions and `src/`

As an analysis grows, repeated computations or transformations may be moved into R functions stored in `src/` and presented as a package. The notebook then focuses on the analytical task and its interpretation, while the function encapsulates a reusable operation.

For example, a simple missingness computation could be defined as:

```
proportion_missing <- function(x) {
  mean(is.na(x))
}

proportion_missing(c(2, NA, 4, NA))
```

```
[1] 0.5
```

This example illustrates separation of responsibility, not a requirement that every one-line operation become a function. Extracting code is worthwhile when it improves clarity, consistency, testing, or reuse.

5.3 Executable scripts and `scripts/`

A `scripts/` directory can hold programs that coordinate tasks: acquiring data, generating derived datasets, validating outputs, or rendering a sequence of documents. In this convention, `src/` contains reusable computational building blocks, while `scripts/` contains entry points that execute a workflow.

This distinction is helpful but not universal. Some projects use different naming conventions or adopt a workflow manager. The important requirement is that each component has a clear responsibility and documented inputs and outputs.

5.4 From interactive analysis to an automated pipeline

A project may develop progressively:

Level	Main components	Main capability
Reproducible analysis	Data, notebooks, reports	Re-execute documented analyses
Modular computation	Add reusable R functions	Reduce duplication and isolate computational logic
Automated workflow	Add orchestration scripts or workflow tools	Run linked operations systematically

These levels represent increasing technical sophistication, not mandatory stages. A small research project may be fully adequate at the first level.

An automated workflow should make dependencies explicit: which inputs are required, which process produces each output, and what needs to be rerun after an input changes. For complex projects, workflow tools can automate dependency tracking; for smaller projects, clear instructions and a few well-designed scripts may suffice.

5.5 Reproducibility checks

Before considering an analysis reproducible, verify that:

1. The original inputs can be accessed and identified.
2. The project explains how to install or restore the required software environment.
3. Notebooks run from a clean session in a documented order, or independently where intended.
4. Dependencies between notebooks, scripts, and generated datasets are explicit.
5. Important outputs can be regenerated without undocumented manual edits.

6. Reports agree with the computational outputs that support them.
7. Methodological decisions and limitations are recorded in an appropriate location.

These checks connect the computational and methodological dimensions of reproducibility. A successful render is necessary evidence for computational reproducibility, but it does not by itself establish that the analysis was methodologically appropriate.

6 Educational Use of the Framework

6.1 Instructor responsibilities and intended competencies

The framework is intended to guide instructors in designing and supervising research-oriented Data Science projects. Its conventions are not merely file-management rules: each creates opportunities for students to practice aspects of research work.

Practice	Intended competency or habit
Clone and maintain a shared repository	Version control, collaboration, and traceability
Restore an `renv` environment	Awareness and management of computational dependencies
Separate raw and processed data	Data provenance and disciplined preparation
Develop task-focused notebooks	Computational documentation and analytical modularity
Execute notebooks in clean sessions	Verification of computational reproducibility
Prepare research reports	Methodological reasoning, justification, and research continuity
Develop a scientific paper when warranted	Positioning, substantiating, and communicating a research contribution

These are *intended learning outcomes*, not claims that the practices automatically produce or demonstrate competence. Instructors should determine what evidence of student learning is appropriate in their course context. A student may begin practicing these habits by following a supplied project template before understanding every underlying principle; the instructor should understand why the conventions matter.

6.2 Framework versus classroom implementation

The repository architecture describes how research materials are organized; the research methodology describes how the investigation proceeds; and the instructional design describes

how students learn to participate in that work. These structures need not correspond one-to-one.

For this reason, the present guide does not prescribe course stages, grading rubrics, or step-by-step student instructions. Those may be developed separately, based on students' preparation and the requirements of a particular course. Similarly, publishing an educational account of the framework would require a distinct argument and evidence about its implementation and educational contribution.

7 Concluding Principles

A reproducible Data Science project benefits from a separation of responsibilities:

- *Computational notebooks* explain *how* coherent analytical tasks are executed, following the philosophy of literate programming.
- *Research reports* preserve *what* was investigated and *why* consequential methodological decisions were made, enabling evaluation and research continuity.
- *Scientific papers* communicate selected findings, establish their scientific context, and provide evidence and methodological detail sufficient to scrutinize the contributions claimed.
- *Reusable functions* implement computational operations that benefit from being shared.
- *Scripts and workflow tools* coordinate repeatable execution when the project requires automation.
- *The repository* connects research materials, inputs, dependencies, and outputs into a traceable and reproducible whole.

The objective is not to impose a complex software architecture or publication workflow on every investigation. It is to preserve methodological knowledge, make computations reproducible, and support responsible scientific communication, introducing additional structure only when it improves clarity, reliability, collaboration, or maintainability.